My Project

Generated by Doxygen 1.10.0

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Circuit . . . . . **??**
  - Gate . . . . . **??**
    - AndGate . . . . . **??**
    - NotGate . . . . . **??**
    - OrGate . . . . . **??**
- Port . . . . . **??**
- QMainWindow
  - MainWindow . . . . . **??**
- QOpenGLFunctions
  - Canvas . . . . . **??**
- QOpenGLWidget
  - Canvas . . . . . **??**
- Spritesheet . . . . . **??**
- Wire . . . . . **??**

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AndGate . . . . . ??
Canvas
This class provides an interactive graphical workspace for circuit components . . . . . ??
Circuit . . . . . ??
Gate . . . . . ??
MainWindow . . . . . ??
NotGate . . . . . ??
OrGate . . . . . ??
Port . . . . . ??
Spritesheet . . . . . ??
Wire . . . . . ??

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

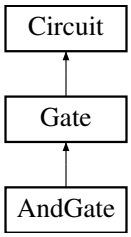
canvas.h
This declares the interactive circuit workspace . . . . . ??
circuit.h . . . . . ??
gate.h . . . . . ??
gates.h . . . . . ??
mainwindow.h . . . . . ??
port.h . . . . . ??
spritesheet.h . . . . . ??
wire.h . . . . . ??

Chapter 4

Class Documentation

4.1 AndGate Class Reference

Inheritance diagram for AndGate:



Public Member Functions

- void draw (QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom) override
- bool contains (double worldX, double worldY) const

Public Member Functions inherited from Gate

- double **getX** ()
- double **getY** ()
- const std::vector< Port * > & **getInputPorts** () const
- const std::vector< Port * > & **getOutputPorts** () const

Public Member Functions inherited from Circuit

- double **getX** () const
- double **getY** () const
- void **setPosition** (double newX, double newY)

Additional Inherited Members

Protected Attributes inherited from Gate

- int **numInputs** = 0
- int **numOutputs** = 1
- double **width** = 50.0
- double **height** = 50.0
- std::vector< Port * > **inputPorts**
- std::vector< Port * > **outputPorts**

Protected Attributes inherited from Circuit

- double **x** = 0
- double **y** = 0

4.1.1 Member Function Documentation

4.1.1.1 contains()

```
bool AndGate::contains (
            double worldX,
            double worldY ) const  [inline], [virtual]
```

Implements Circuit.

4.1.1.2 draw()

```
void AndGate::draw (
            QPainter & painter,
            Spritesheet & sprites,
            double screenX,
            double screenY,
            double zoom )  [inline], [override], [virtual]
```

Implements Circuit.

The documentation for this class was generated from the following file:

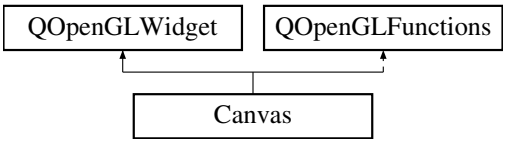
- gates.h

4.2 Canvas Class Reference

This class provides an interactive graphical workspace for circuit components.

`#include <canvas.h>`

Inheritance diagram for Canvas:



Public Member Functions

- Canvas (QWidget *parent=NULL)
  Canvas constructor.
- void **drawGrid** (QPainter &)
- double **screenToWorldX** (double) const
- double **screenToWorldY** (double) const
- double **worldToScreenX** (double) const
- double **worldToScreenY** (double) const
- double **snapToGrid** (double)
- void **updateCamera** ()
- void **placeGate** (Gate *)
- Port * **findport** (double, double)

Protected Member Functions

- void **initializeGL** () override
- void **paintGL** () override
- void **resizeGL** (int width, int height) override
- void **drawPort** (QPainter &, Port *)
- void **drawWire** (QPainter &, Wire *)
- void **drawTempWire** (QPainter &)
- void **mousePressEvent** (QMouseEvent *event) override
- void **mouseMoveEvent** (QMouseEvent *event) override
- void **mouseReleaseEvent** (QMouseEvent *event) override
- void **wheelEvent** (QWheelEvent *event) override
- void **keyPressEvent** (QKeyEvent *event) override
- void **keyReleaseEvent** (QKeyEvent *event) override

4.2.1 Detailed Description

This class provides an interactive graphical workspace for circuit components.

Canvas is responsible for displaying and managing the GUI for the workspace of the logic simulator

The program uses QOpenGLWidget OpenGl rendering and QPainter for drawing the grid and circuit elements

Current Features include:

- Rendering a grid-based workspace
- Displaying AND, OR, and NOT gates
- Dragging a gate using the leftmouse button
- Panning the grid using either the middle mouse button or using the WASD keys
- Zooming in and out toward the mouse cursor

Note

Circuit simulation logic is not yet integrated into the graphical user interface

See also

Circuit

Gate

Gates

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Canvas()

```
Canvas::Canvas (
            QWidget * parent = NULL )
```

Canvas constructor.

Initializes the canvas and for now initializes an AND, OR, and NOT gate

Parameters

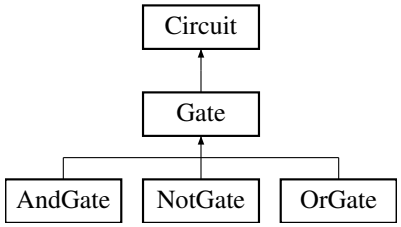
parent	Parent QWidget

The documentation for this class was generated from the following files:

- canvas.h
- canvas.cpp

4.3 Circuit Class Reference

Inheritance diagram for Circuit:



Public Member Functions

- virtual void **draw** (QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom)=0
- virtual bool **contains** (double worldX, double worldY) const =0
- double **getX** () const
- double **getY** () const
- void **setPosition** (double newX, double newY)

Protected Attributes

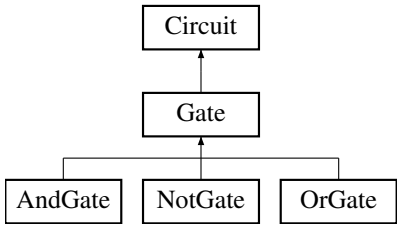
- double **x** = 0
- double **y** = 0

The documentation for this class was generated from the following files:

- circuit.h
- circuit.cpp

4.4 Gate Class Reference

Inheritance diagram for Gate:



Public Member Functions

- double **getX** ()
- double **getY** ()
- const std::vector< Port * > & **getInputPorts** () const
- const std::vector< Port * > & **getOutputPorts** () const

Public Member Functions inherited from Circuit

- virtual void **draw** (QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom)=0
- virtual bool **contains** (double worldX, double worldY) const =0
- double **getX** () const
- double **getY** () const
- void **setPosition** (double newX, double newY)

Protected Attributes

- int **numInputs** = 0
- int **numOutputs** = 1
- double **width** = 50.0
- double **height** = 50.0
- std::vector< Port * > **inputPorts**
- std::vector< Port * > **outputPorts**

Protected Attributes inherited from Circuit

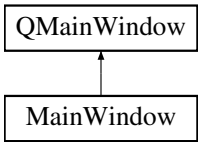
- double **x** = 0
- double **y** = 0

The documentation for this class was generated from the following files:

- gate.h
- gate.cpp

4.5 MainWindow Class Reference

Inheritance diagram for MainWindow:



Public Member Functions

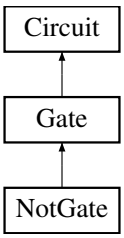
- **MainWindow** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- mainwindow.h
- mainwindow.cpp

4.6 NotGate Class Reference

Inheritance diagram for NotGate:



Public Member Functions

- void draw (QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom) override
- bool contains (double worldX, double worldY) const

Public Member Functions inherited from Gate

- double **getX** ()
- double **getY** ()
- const std::vector< Port * > & **getInputPorts** () const
- const std::vector< Port * > & **getOutputPorts** () const

Public Member Functions inherited from Circuit

- double **getX** () const
- double **getY** () const
- void **setPosition** (double newX, double newY)

Additional Inherited Members

Protected Attributes inherited from Gate

- int **numInputs** = 0
- int **numOutputs** = 1
- double **width** = 50.0
- double **height** = 50.0
- std::vector< Port * > **inputPorts**
- std::vector< Port * > **outputPorts**

Protected Attributes inherited from Circuit

- double **x** = 0
- double **y** = 0

4.6.1 Member Function Documentation

4.6.1.1 contains()

```
bool NotGate::contains (
            double worldX,
            double worldY ) const  [inline], [virtual]
```

Implements Circuit.

4.6.1.2 draw()

```
void NotGate::draw (
            QPainter & painter,
            Spritesheet & sprites,
            double screenX,
            double screenY,
            double zoom )  [inline], [override], [virtual]
```

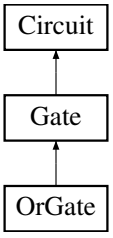
Implements Circuit.

The documentation for this class was generated from the following file:

- gates.h

4.7 OrGate Class Reference

Inheritance diagram for OrGate:



Public Member Functions

- void draw (QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom) override
- bool contains (double worldX, double worldY) const

Public Member Functions inherited from Gate

- double **getX** ()
- double **getY** ()
- const std::vector< Port * > & **getInputPorts** () const
- const std::vector< Port * > & **getOutputPorts** () const

Public Member Functions inherited from Circuit

- double **getX** () const
- double **getY** () const
- void **setPosition** (double newX, double newY)

Additional Inherited Members

Protected Attributes inherited from Gate

- int **numInputs** = 0
- int **numOutputs** = 1
- double **width** = 50.0
- double **height** = 50.0
- std::vector< Port * > **inputPorts**
- std::vector< Port * > **outputPorts**

Protected Attributes inherited from Circuit

- double **x** = 0
- double **y** = 0

4.7.1 Member Function Documentation

4.7.1.1 contains()

```
bool OrGate::contains (
            double worldX,
            double worldY ) const  [inline], [virtual]
```

Implements Circuit.

4.7.1.2 draw()

```
void OrGate::draw (
            QPainter & painter,
            Spritesheet & sprites,
            double screenX,
            double screenY,
            double zoom )  [inline], [override], [virtual]
```

Implements Circuit.

The documentation for this class was generated from the following file:

- gates.h

4.8 Port Class Reference

Public Member Functions

- **Port** (Gate *, double, double, PortType)
- double **getX** () const
- double **getY** () const
- PortType **getType** ()
- bool **contains** (double, double) const

The documentation for this class was generated from the following files:

- port.h
- port.cpp

4.9 Spritesheet Class Reference

Public Member Functions

- **Spritesheet** (const QString &)
- void **drawSprite** (QPainter &, int, int, double, double, double, double)

The documentation for this class was generated from the following files:

- spritesheet.h
- spritesheet.cpp

4.10 Wire Class Reference

Public Member Functions

- **Wire** (Port *, Port *)
- Port * **getStart** () const
- Port * **getEnd** () const

The documentation for this class was generated from the following files:

- wire.h
- wire.cpp

Chapter 5

File Documentation

5.1 canvas.h File Reference

This declares the interactive circuit workspace.

```
#include <QOpenGLWidget>
#include <QOpenGLFunctions>
#include <QPainter>
#include <QMouseEvent>
#include <QWheelEvent>
#include <QKeyEvent>
#include <QTimer>
#include <QCursor>
#include <cmath>
#include <algorithm>
#include <vector>
#include "port.h"
#include "spritesheet.h"
```

Classes

- class Canvas

  This class provides an interactive graphical workspace for circuit components.

5.1.1 Detailed Description

This declares the interactive circuit workspace.

Author

Ethan Graham

Date

10/16/2026

5.2 canvas.h

Go to the documentation of this file.

```
100001
100036 #ifndef CANVAS_H
100037 #define CANVAS_H
100038
100039 #include <QOpenGLWidget>
100040 #include <QOpenGLFunctions>
100041 #include <QPainter>
100042 #include <QMouseEvent>
100043 #include <QWheelEvent>
100044 #include <QKeyEvent>
100045 #include <QTimer>
100046 #include <QCursor>
100047
100048 #include <cmath>
100049 #include <algorithm>
100050 #include <vector>
100051
100052 #include "port.h"
100053 #include "spritesheet.h"
100054
100055 class Wire;
100056 class Port;
100057
100058 class Canvas : public QOpenGLWidget,
100059                protected QOpenGLFunctions
100060 {
100061     Q_OBJECT
100062
100063 public:
100072     Canvas(QWidget * parent = NULL);
100073
100074
100078     void drawGrid(QPainter&);
100079
100083     double screenToWorldX(double) const;
100084     double screenToWorldY(double) const;
100085
100086     double worldToScreenX(double) const;
100087     double worldToScreenY(double) const;
100088
100089     double snapToGrid(double);
100090
100094     // ---- MOVE TO CAMERA CLASS LATER ----
100095     void updateCamera();
100096     // ---- MOVE TO CAMERA CLASS LATER ----
100097
100101     void placeGate(Gate*);
100102
100106     Port *findport(double, double);
100107
100108 protected:
100109     void initializeGL() override;
100110     void paintGL() override;
100111     void resizeGL(int width, int height) override;
100112     void drawPort(QPainter&, Port*);
100113     void drawWire(QPainter&, Wire*);
100114     void drawTempWire(QPainter&);
100115
100116     // Control Events
100117     void mousePressEvent(QMouseEvent *event) override;
100118     void mouseMoveEvent(QMouseEvent *event) override;
100119     void mouseReleaseEvent(QMouseEvent *event) override;
100120     void wheelEvent(QWheelEvent *event) override;
100121     void keyPressEvent(QKeyEvent *event) override;
100122     void keyReleaseEvent(QKeyEvent *event) override;
100123
100124 private:
100125     Spritesheet spriteSheet;
100126
100127     // ---- MOVE TO CAMERA CLASS LATER ----
100128     double zoom = 1.0;
100129     double cameraX = 0.0;
100130     double cameraY = 0.0;
100131
100132     QTimer cameraTimer;
100133     // ---- MOVE TO CAMERA CLASS LATER ----
100134
100135     const double gridsize = 25;
100136
100137     bool panning = false;
100138     bool draggingGate = false;
100139     Gate *selectedGate = NULL;
```

```
100140     QPoint lastMousePosition;
100141     QPoint dragOffset;
100142
100143     bool wPressed = false;
100144     bool aPressed = false;
100145     bool sPressed = false;
100146     bool dPressed = false;
100147
100148     Port *wireStart = NULL;
100149     bool drawingWire = false;
100150
100151     std::vector<Gate*> gates;
100152     std::vector<Wire*> wires;
100153
100154     QPointF mousePosition;
100155 };
100156
100157 #endif // CANVAS_H
```

5.3 circuit.h

```
100001 #ifndef CIRCUIT_H
100002 #define CIRCUIT_H
100003
100004 #include "spritesheet.h"
100005
100006 #include <QPainter>
100007
100008 class Circuit
100009 {
100010 public:
100011     //Circuit();
100012     virtual ~Circuit() = default;
100013
100014     //virtual void draw(QPainter &painter, double screenX, double screenY, double zoom) = 0;
100015     virtual void draw(QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double
zoom) = 0;
100016     virtual bool contains(double worldX, double worldY) const = 0;
100017
100018     double getX() const;
100019     double getY() const;
100020
100021     void setPosition(double newX, double newY);
100022
100023 protected:
100024     double x = 0;
100025     double y = 0;
100026 };
100027
100028 #endif // CIRCUIT_H
```

5.4 gate.h

```
100001 #ifndef GATE_H
100002 #define GATE_H
100003
100004 #include "circuit.h"
100005 #include <vector>
100006 #include <iostream>
100007
100008 class Port;
100009
100010 class Gate : public Circuit
100011 {
100012 public:
100013     virtual ~Gate() = default;
100014
100015     double getX();
100016     double getY();
100017
100018     const std::vector<Port*>& getInputPorts() const;
100019     const std::vector<Port*>& getOutputPorts() const;
100020
100021 protected:
100022     int numInputs = 0;
100023     int numOutputs = 1;
100024
100025     double width = 50.0;
100026     double height = 50.0;
```

```
100027
100028     std::vector<Port*> inputPorts;
100029     std::vector<Port*> outputPorts;
100030 };
100031
100032 #endif // GATE_H
```

5.5 gates.h

```
100001 #ifndef GATES_H
100002 #define GATES_H
100003
100004 #include "gate.h"
100005 #include "port.h"
100006 #include "spritesheet.h"
100007
100008 class AndGate : public Gate
100009 {
100010 public:
100011     AndGate()
100012     {
100013         numInputs = 2;
100014
100015         inputPorts.push_back(new Port(this, -width / 2, -12, PortType::INPUT));
100016         inputPorts.push_back(new Port(this, -width / 2, 12, PortType::INPUT));
100017         outputPorts.push_back(new Port(this, width / 2, 0, PortType::OUTPUT));
100018     }
100019
100020     void draw(QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom)
override
100021     {
100022         double drawWidth = width * zoom;
100023         double drawHeight = height * zoom;
100024
100025         sprites.drawSprite(painter, 0, 0,
100026                            screenX - drawWidth / 2, screenY - drawHeight / 2,
100027                             drawWidth, drawHeight);
100028
100029         QPen gridPen(QColor(0, 0, 250));
100030
100031         painter.setPen(gridPen);
100032
100033         double screenWidth = width * zoom;
100034         double screenHeight = height * zoom;
100035
100036         painter.drawRect(screenX - screenWidth / 2.0,
100037                          screenY - screenHeight / 2.0,
100038                          screenWidth, screenHeight);
100039     }
100040
100041     bool contains(double worldX, double worldY) const
100042     {
100043         return worldX >= x - width / 2.0 &&
100044                worldX <= x + width / 2.0 &&
100045                worldY >= y - height / 2.0 &&
100046                worldY <= y + height / 2.0;
100047     }
100048 };
100049
100050 class OrGate : public Gate
100051 {
100052 public:
100053     OrGate()
100054     {
100055         numInputs = 2;
100056
100057         inputPorts.push_back(new Port(this, -width / 2, -12, PortType::INPUT));
100058         inputPorts.push_back(new Port(this, -width / 2, 12, PortType::INPUT));
100059         outputPorts.push_back(new Port(this, width / 2, 0, PortType::OUTPUT));
100060     }
100061
100062     void draw(QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom)
override
100063     {
100064         double drawWidth = width * zoom;
100065         double drawHeight = height * zoom;
100066
100067         sprites.drawSprite(painter, 0, 1,
100068                            screenX - drawWidth / 2, screenY - drawHeight / 2,
100069                            drawWidth, drawHeight);
100070
100071         QPen gridPen(QColor(0, 250, 0));
100072
```

```
100073         painter.setPen(gridPen);
100074
100075         double screenWidth = width * zoom;
100076         double screenHeight = height * zoom;
100077
100078         painter.drawRect(screenX - screenWidth / 2.0,
100079                          screenY - screenHeight / 2.0,
100080                          screenWidth, screenHeight);
100081     }
100082
100083     bool contains(double worldX, double worldY) const
100084     {
100085         return worldX >= x - width / 2.0 &&
100086                worldX <= x + width / 2.0 &&
100087                worldY >= y - height / 2.0 &&
100088                worldY <= y + height / 2.0;
100089     };
100090 };
100091
100092 class NotGate : public Gate
100093 {
100094 public:
100095     NotGate()
100096     {
100097         numInputs = 1;
100098
100099         inputPorts.push_back(new Port(this, -width / 2, 0, PortType::INPUT));
100100         outputPorts.push_back(new Port(this, width / 2, 0, PortType::OUTPUT));
100101     }
100102
100103     void draw(QPainter &painter, Spritesheet &sprites, double screenX, double screenY, double zoom)
override
100104     {
100105         double drawWidth = width * zoom;
100106         double drawHeight = height * zoom;
100107
100108         sprites.drawSprite(painter, 0, 2,
100109                            screenX - drawWidth / 2, screenY - drawHeight / 2,
100110                            drawWidth, drawHeight);
100111
100112         QPen gridPen(QColor(250, 0, 0));
100113
100114         painter.setPen(gridPen);
100115
100116         double screenWidth = width * zoom;
100117         double screenHeight = height * zoom;
100118
100119         painter.drawRect(screenX - screenWidth / 2.0,
100120                          screenY - screenHeight / 2.0,
100121                          screenWidth, screenHeight);
100122     }
100123
100124     bool contains(double worldX, double worldY) const
100125     {
100126         return worldX >= x - width / 2.0 &&
100127                worldX <= x + width / 2.0 &&
100128                worldY >= y - height / 2.0 &&
100129                worldY <= y + height / 2.0;
100130     }
100131 };
100132
100133 #endif // GATES_H
```

5.6 mainwindow.h

```
100001 #ifndef MAINWINDOW_H
100002 #define MAINWINDOW_H
100003
100004 #include <QMainWindow>
100005
100006 QT_BEGIN_NAMESPACE
100007 namespace Ui {
100008 class MainWindow;
100009 }
100010 QT_END_NAMESPACE
100011
100012 class MainWindow : public QMainWindow
100013 {
100014     Q_OBJECT
100015
100016 public:
100017     MainWindow(QWidget *parent = nullptr);
100018     ~MainWindow();
```

```
100019
100020
100021 private:
100022     Ui::MainWindow *ui;
100023 };
100024 #endif // MAINWINDOW_H
```

5.7 port.h

```
100001 #ifndef PORT_H
100002 #define PORT_H
100003
100004 class Gate;
100005
100006 enum class PortType
100007 {
100008     INPUT,
100009     OUTPUT
100010 };
100011
100012 class Port
100013 {
100014 public:
100015     Port(Gate*, double, double, PortType);
100016
100017     double getX() const;
100018     double getY() const;
100019
100020     PortType getType();
100021
100022     bool contains(double, double) const;
100023 private:
100024     Gate *parentGate;
100025
100026     double offsetX;
100027     double offsetY;
100028
100029     PortType type;
100030 };
100031
100032 #endif // PORT_H
```

5.8 spritesheet.h

```
100001 #ifndef SPRITESHEET_H
100002 #define SPRITESHEET_H
100003
100004 #include <QPixmap>
100005 #include <QPainter>
100006
100007 class Spritesheet
100008 {
100009 public:
100010     Spritesheet(const QString &);
100011
100012     void drawSprite(QPainter&, int, int, double, double, double, double);
100013 private:
100014     QPixmap spriteSheet;
100015
100016     int spriteWidth = 64;
100017     int spriteHeight = 64;
100018 };
100019
100020 #endif // SPRITESHEET_H
```

5.9 wire.h

```
100001 #ifndef WIRE_H
100002 #define WIRE_H
100003
100004 class Port;
100005
100006 class Wire
100007 {
100008 public:
```

```
100009     Wire(Port*, Port*);
100010
100011     Port *getStart() const;
100012     Port *getEnd() const;
100013 private:
100014     Port *start;
100015     Port *end;
100016 };
100017
100018 #endif // WIRE_H
```